Interactive 3d Pinball Game

Computer Graphics and Image Processing Laboratory

CSE 4102

Submitted by

Al Nahian Zarif

Roll: 2107098

4th Year

Project Overview

This C++17/OpenGL 3.3 application combines a playable pinball machine with an interactive gaming lounge. The player launches a steel ball, operates two flippers, hits six bumpers and four drop targets, collects score and bonus, and manages three balls. The lounge adds movable cameras, independent lighting and fan controls, a hinged refrigerator door, and CPU ray tracing. Procedural meshes, transformations, lighting, animation and collision response share the same scene state. The report uses the original source inspection, application captures and validation results; numerical values are machine coordinates unless stated otherwise.

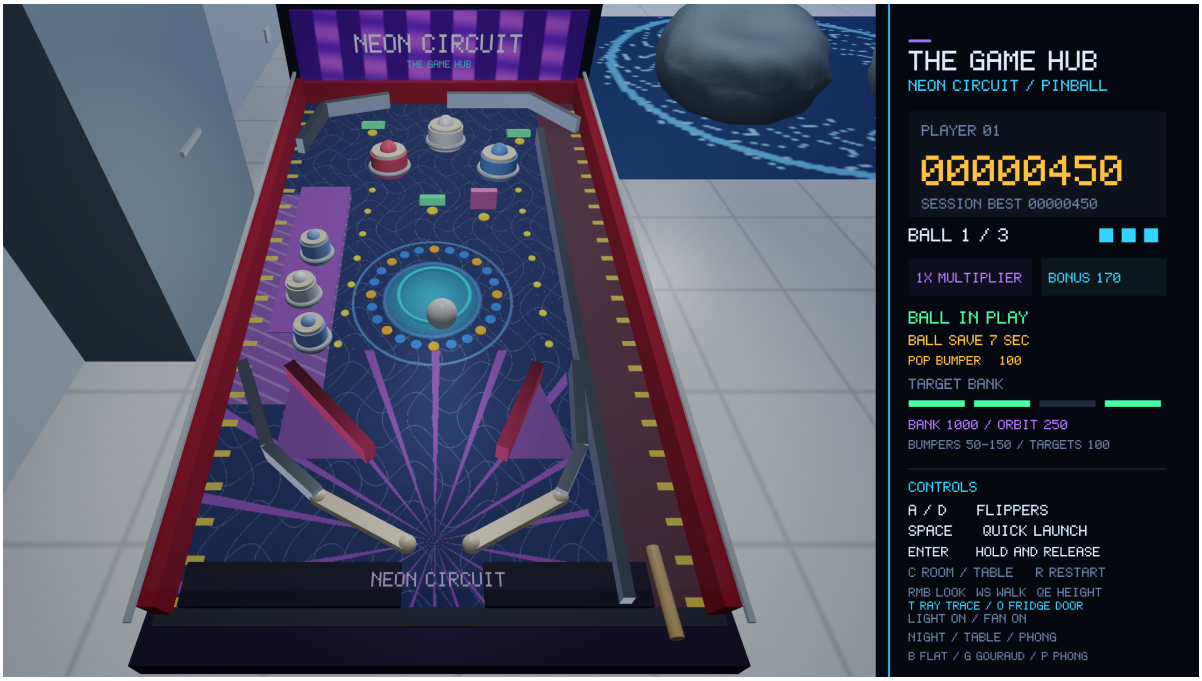


Figure 1.1: Actual pinball gameplay capture showing the playfield, ball and score console.

Implemented Features

Mesh.cpp generates primitives and custom geometry; Transform.cpp composes object models. PinballMachine.cpp and Ball.cpp implement fixed-step gameplay, while GameHub.cpp builds and animates the lounge. GLSL programs provide selectable Flat, Gouraud and Phong modes. RayTracer.cpp and RayTraceView.cpp supply BVH-accelerated ray transport and background rendering; main.cpp and HUD.cpp handle input and consoles.

Gaming Lounge

The room contains tiled flooring, navy panels, LED borders, a television and console, two beanbags on a rug, cabinets, a beverage refrigerator, pendant lights, a fan, a mascot, an air conditioner and a glass wall. The game console displays score, session best, ball count, multiplier, bonus, target-bank progress and save time. Captures show representative output; tests establish the specific behaviors they exercise.



Figure 2.1: Actual gaming-lounge capture with room controls and the pinball machine at the side.

Coordinates and Mathematical Foundation

The right-handed coordinate system uses Z as vertical. Machine X spans the board width and Y points toward the backbox. Mesh positions begin in local space; an object model places each mesh, and a separate scene transform places the complete machine in the room. Room scenery uses the identity scene transform.

$$p_{local} = (x, y, z, 1)^{T} ; p_{world} = M_{scene} M_{object} p_{local} \tag{3.1}$$

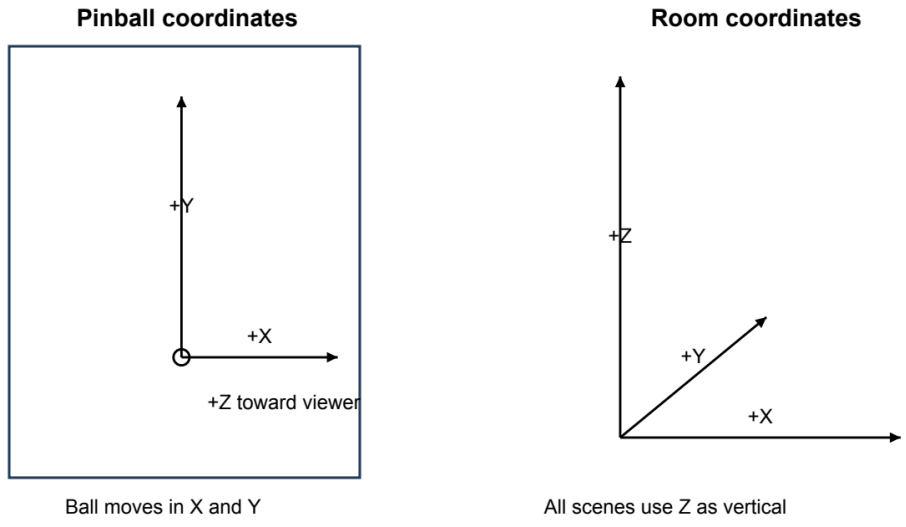


Figure 3.1: Coordinate conventions for the machine and the room.

Points, Directions and Vector Operations

Homogeneous points use w = 1 and receive translation; directions use w = 0. Dot products measure projections and angular alignment for contact and lighting. Cross products construct camera bases, surface normals and rail sides. Normalization follows a zero-length check. Scene units are mathematical units; the lounge scale changes presentation without converting the physics model.

$$\text{point} = (x, y, z, 1)^T \; ; \; \text{direction} = (d_x, d_y, d_z, 0)^T \tag{3.2}$$

$$a \cdot b = a_x b_x + a_y b_y + a_z b_z \tag{3.3}$$

$$|v| = \sqrt{(v \cdot v)} \; ; \; \hat{v} = v / |v| \tag{3.4}$$

Table 3.1: Coordinate and numerical conventions.

Quantity	Value or convention
Playfield width and projected length	7.0 and 12.0 simulation units
Playfield center height and tilt	1.4 units and 8 degrees
Ball radius	0.28 simulation units
Room size	12 x 14 x 5.6 room units
Machine room placement	Origin (-3.5, -3.8, 0.45); uniform scale 0.28
Angles in object state	Degrees; converted to radians for trigonometry and GLM
Vector safeguards	Zero-length and near-parallel cases are checked before division

Transformation Mathematics

Matrices below are written by rows and act on column vectors; GLM stores their data in column-major order. Translation occupies the final column. Scaling changes each axis independently; the lounge machine uses uniform scale 0.28.

$$T = \begin{bmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{4.1}$$

$$x' = x + t_x \; ; \; y' = y + t_y \; ; \; z' = z + t_z \tag{4.2}$$

$$S = \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{4.3}$$

Rotation

Positive rotations follow the right-hand rule. The board rotates about X; the flippers, fan and door rotate about Z. General object rotation uses glm::quat(glm::radians(rotation)), converted to a 4 x 4 matrix.

$$R_x = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta & 0 \\ 0 & \sin\theta & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{4.4}$$

$$R_y = \begin{bmatrix} \cos\theta & 0 & \sin\theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin\theta & 0 & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{4.5}$$

$$R_z = \begin{bmatrix} \cos\theta & -\sin\theta & 0 & 0 \\ \sin\theta & \cos\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{4.6}$$

Object Model, Machine Placement and Board Tilt

Transform::getModelMatrix composes translation, quaternion rotation and scale, with the rightmost operation acting first. The board uses width 7 and local length 12 / cos 8°, so its tilted projection spans Y = ±6. Its surface height is 0.556755 at Y = -6, 1.4 at the center and 2.243245 at Y = +6.

$$M_{object} = T\, R_{quat}\, S \tag{4.7}$$

$$M_{machine} = T(-3.5, -3.8, 0.45)\, S(0.28, 0.28, 0.28) \tag{4.8}$$

$$p_{room} = (-3.5, -3.8, 0.45) + 0.28\, p_{machine} \tag{4.9}$$

$$M_{board} = T(0, 0, 1.4)\, R_x(8°) \tag{4.10}$$

$$z(y) = 1.4 + y \tan(8°) \tag{4.11}$$

Cuboid and Leg Coordinates

The cabinet has size (7.8,13.3,0.4) and center (0,0,0.22). Leg size is (0.15,0.15,0.5), with centers (±3.5,±5.7,0.25). Adding half-dimension sign combinations yields all corners. For the front-left leg, local v0 = (-0.075,-0.075,-0.25) becomes (-3.575,-5.775,0) in machine space and (-4.501,-5.417,0.45) after lounge placement.

$$\text{corner} = \text{center} + (\pm w/2, \pm h/2, \pm d/2) \tag{4.12}$$

Procedural Geometry

A Vertex stores position, normal, RGB color and UV coordinates. Expanded meshes store three records per triangle; indexed meshes share records through an EBO. Unique geometric corners therefore differ from GPU record counts. Cuboids have eight corners, twelve triangles and 36 expanded records, with separate normals on each face for hard edges.

Table 5.1: All eight unique local cuboid corners.

ID	General position	Unit cube position
v0	(-w/2, -h/2, -d/2)	(-0.5, -0.5, -0.5)
v1	(w/2, -h/2, -d/2)	(0.5, -0.5, -0.5)
v2	(w/2, h/2, -d/2)	(0.5, 0.5, -0.5)
v3	(-w/2, h/2, -d/2)	(-0.5, 0.5, -0.5)
v4	(-w/2, -h/2, d/2)	(-0.5, -0.5, 0.5)
v5	(w/2, -h/2, d/2)	(0.5, -0.5, 0.5)
v6	(w/2, h/2, d/2)	(0.5, 0.5, 0.5)
v7	(-w/2, h/2, d/2)	(-0.5, 0.5, 0.5)

Cuboid Faces and Triangle Assembly

The factory preserves its face labels and assigned normals. Some triangle windings disagree with those normals; raster face culling is disabled, and the ray exporter aligns geometric normals with vertex-normal sums. In this Z-up scene, the source Back face is physically upper. Face UVs are (0,0), (1,0), (1,1), (0,1), expanded as 0-1-2 and 0-2-3.

Table 5.2: Exact cuboid face order from createCube.

Face	Ordered corners	Assigned normal	Triangles
Front	0, 1, 2, 3	(0, 0, -1)	0-1-2; 0-2-3
Back	4, 7, 6, 5	(0, 0, 1)	4-7-6; 4-6-5
Left	0, 4, 7, 3	(-1, 0, 0)	0-4-7; 0-7-3
Right	1, 5, 6, 2	(1, 0, 0)	1-5-6; 1-6-2
Bottom	0, 4, 5, 1	(0, -1, 0)	0-4-5; 0-5-1
Top	3, 7, 6, 2	(0, 1, 0)	3-7-6; 3-6-2

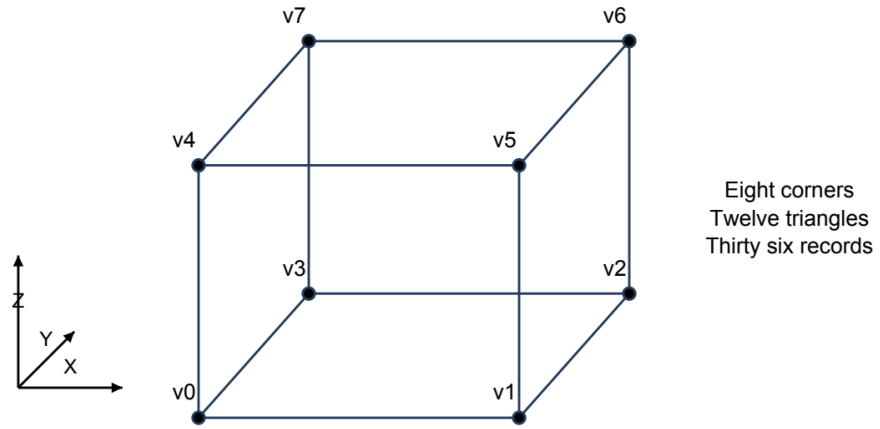


Figure 5.1: The eight cuboid corner labels used by the source.

Plane and Inclined Board

createPlane places four corners in XY at Z = 0 and expands triangles (0,1,2), (0,2,3), all with normal (0,0,1). The playfield repeats its artwork once; the 12 x 14 room floor uses repeat 3.5. The tilted normal is (0,-sin 8°,cos 8°). Ball seating adds radius vertically to the board height.

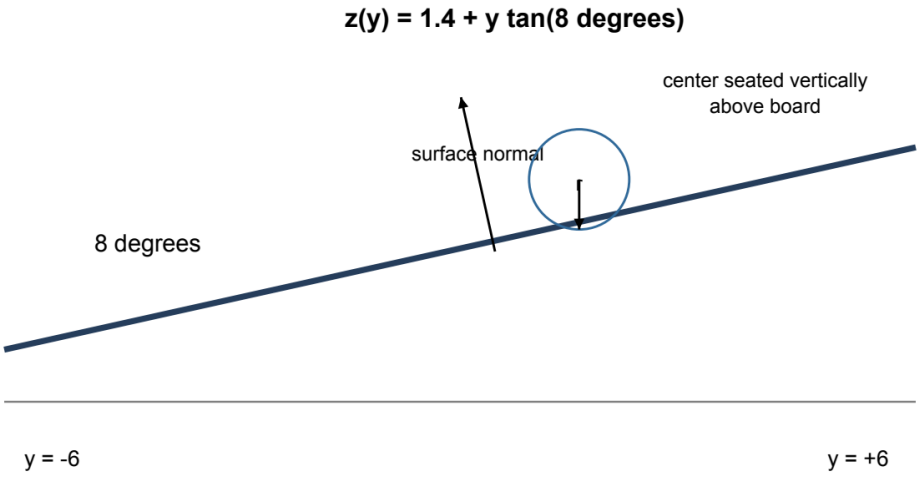


Figure 5.2: Inclined board surface and the vertically seated ball center.

$$N_{board} = (0, -0.139173, 0.990268) \tag{5.1}$$

Table 5.3: Plane positions and texture coordinates.

Corner	Local position	UV for repeat q
v0	(-w/2, -d/2, 0)	(0, 0)
v1	(w/2, -d/2, 0)	(q, 0)
v2	(w/2, d/2, 0)	(q, q)
v3	(-w/2, d/2, 0)	(0, q)

Cylinder

createCylinder uses local Z as its axis, two rings at Z = ±h/2, radial side normals and axial cap normals. Each of N sectors contributes two side triangles and one triangle per cap. Side UVs use (i/N,0) and (i/N,1); cap UVs retain zero values. The main bumper uses r = 0.42, h = 0.45 and N = 28, giving 336 records and 112 triangles. Other cylinders form the plunger, fan hub, bottles and lamps.

$$\theta_i = 2\pi i/N \; ; \; x_i = r \cos \theta_i \; ; \; y_i = r \sin \theta_i \tag{5.2}$$

$$p_{i,\pm} = (r \cos \theta_i, \; r \sin \theta_i, \; \pm h/2) \tag{5.3}$$

$$N_{side} = (\cos \theta, \sin \theta, 0) \; ; \; N_{caps} = (0, 0, \pm 1) \tag{5.4}$$

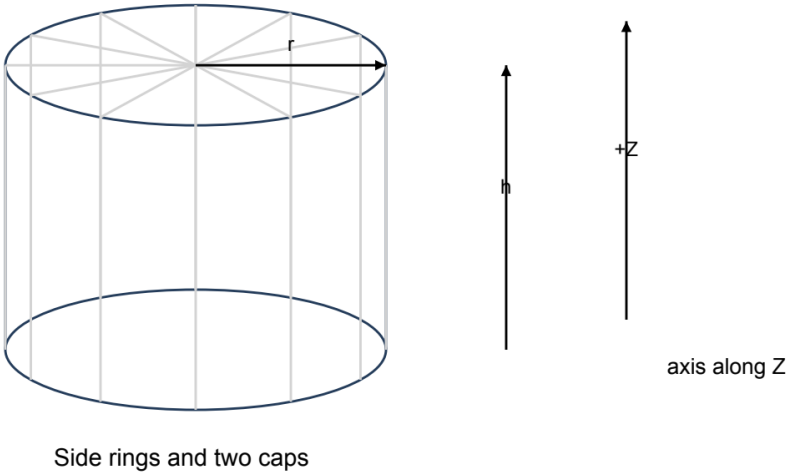


Figure 5.3: Cylinder rings, caps, radius and local Z axis.

$$V = 12N \; ; \; T = 4N \; ; \; \text{EBO entries} = 0 \tag{5.5}$$

Sphere and Steel Ball

Latitude spans -π/2 to +π/2 and longitude spans a full turn. Endpoint rows duplicate the longitude seam and pole positions. The steel ball uses r = 0.28, L = 18 and K = 28; local positions determine normals and angular fractions determine UVs.

$$\varphi_i = -\pi/2 + \pi i/L \; ; \; \theta_j = 2\pi j/K \tag{5.6}$$

$$p(\varphi,\theta) = (r \cos \varphi \cos \theta, \; r \cos \varphi \sin \theta, \; r \sin \varphi) \tag{5.7}$$

$$N = p/r \; ; \; UV = (j/K, \; i/L) \tag{5.8}$$

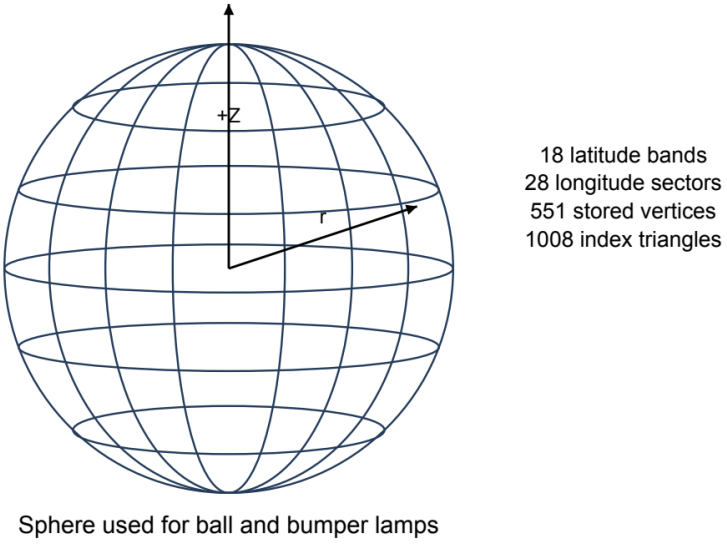


Figure 5.4: Latitude and longitude construction of the steel-ball sphere.

Sphere Indices

Each grid cell contributes two indexed triangles. The ball stores 551 vertices and submits 1008 triangles; 56 pole triangles have zero area, leaving 952 for ray tracing. RayScene rejects a triangle when its squared cross-product length is below 10^{-16}.

$$a = i(K + 1) + j \; ; \; b = a + K + 1 \tag{5.9}$$

$$\text{triangle } 1 = (a,b,a+1) \; ; \; \text{triangle } 2 = (a+1,b,b+1) \tag{5.10}$$

$$V = (L+1)(K+1) = 19 \times 29 = 551 \tag{5.11}$$

$$I = 6LK = 3024 \; ; \; T_{submitted} = 2LK = 1008 \tag{5.12}$$

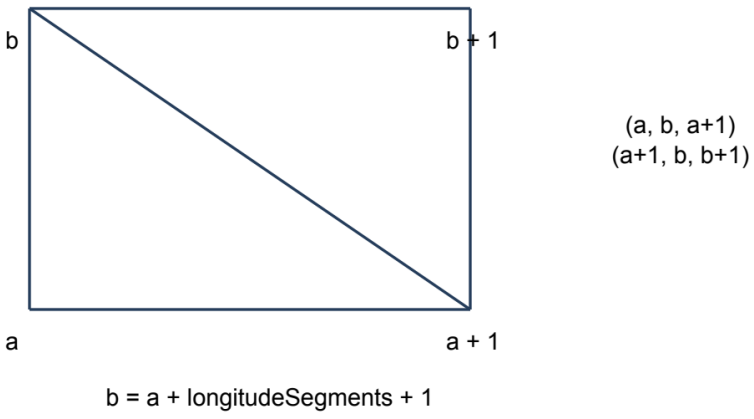


Figure 5.5: Two indexed triangles from one sphere grid cell.

Flipper Mesh

The body is a (1.48,0.22,0.16) cuboid with 36 records. Two upper cap patches centered at X = ±0.74 use 12 latitude intervals and 12 longitude sectors each, adding 1728 records. Total storage is 1764 records and 588 triangles. These are upper hemispherical patches in Z; gameplay contact instead uses a separate 2D capsule around the pivot-to-tip segment.

$$r_{cap} = \sqrt{(0.11^2 + 0.08^2)} = 0.136015 \tag{5.13}$$

$$p = (x_{cap} + s\, r_{cap} \cos\varphi \cos\theta,\ r_{cap} \cos\varphi \sin\theta,\ r_{cap} \sin\varphi) \tag{5.14}$$

$$N = (s \cos\varphi \cos\theta,\ \cos\varphi \sin\theta,\ \sin\varphi) \tag{5.15}$$

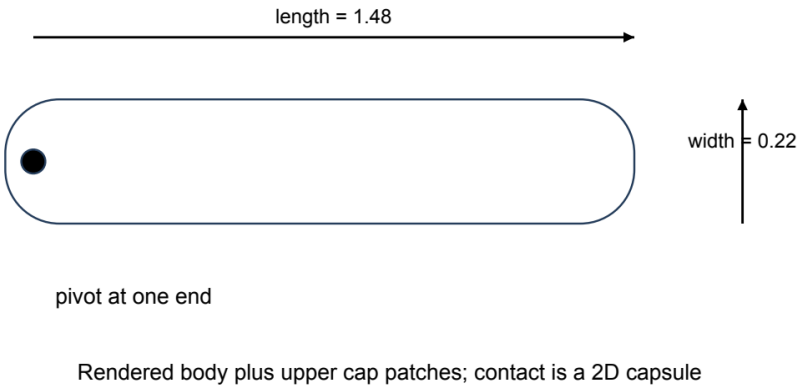


Figure 5.6: Flipper dimensions and the relationship between rendered geometry and contact geometry.

Torus Ring

The reactor ring uses major radius R = 0.68 and tube radius r = 0.04. Sampling 40 major-angle sectors and eight tube sectors, with both seam endpoints, stores (N+1)(M+1) = 369 vertices and 6NM = 1920 indices, or 640 triangles. Cell indexing follows the sphere grid.

$$\alpha_i = 2\pi i/N\ ;\ \beta_j = 2\pi j/M \tag{5.16}$$

$$p = ((R + r\cos\beta)\cos\alpha,\ (R + r\cos\beta)\sin\alpha,\ r\sin\beta) \tag{5.17}$$

$$N = (\cos\alpha\cos\beta,\ \sin\alpha\cos\beta,\ \sin\beta) \tag{5.18}$$

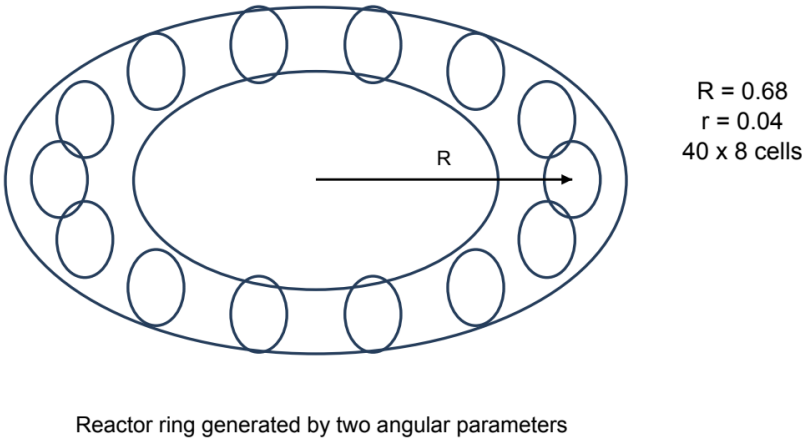


Figure 5.7: Torus geometry used by the reactor insert and bumper skirts.

Spring, Bitmap Text and Television Quad

createSpring is available but unused in the current scene. It samples $t_0 = i/(S-1)$, $t_1 = (i+1)/(S-1)$, forms $\tau = c(t_1)-c(t_0)$, $n = \text{normalize}(\tau \times Z)$ and $b = \text{normalize}(n \times \tau)$, then offsets an eight-sided tube by $0.1r(\cos \beta\, b + \sin \beta\, n)$. The loop $i < S$ reaches $t = S/(S-1)$ at its last endpoint. Bitmap text uses a 5 x 7 font: each active pixel becomes a four-vertex, two-triangle XZ quad facing -Y. Pixel size is textHeight/7; glyph/space advances are six/four pixels. The television is an indexed 2.98 x 1.77 quad with direct UVs and indices (0,1,2,0,2,3).

$$c(t) = (r\cos(2\pi kt),\ r\sin(2\pi kt),\ -h/2 + ht) \tag{5.19}$$

$$V_{spring} = 6 \times 8 \times S = 48S\ ;\ T = 16S \tag{5.20}$$

Table 5.8: Television-screen quad from createScreenMesh(2.98,1.77).

ID	Position	Normal	UV
0	(-1.49, 0, -0.885)	(0,-1,0)	(0,0)
1	(1.49, 0, -0.885)	(0,-1,0)	(1,0)
2	(1.49, 0, 0.885)	(0,-1,0)	(1,1)
3	(-1.49, 0, 0.885)	(0,-1,0)	(0,1)

Beanbags and Rails

Beanbags deform a spherical grid with $a = \pi i/24$, $b = 2\pi j/40$, $x = \sin a \cos b$, $y = \sin a \sin b$ and $z = \cos a$. For $z > 0$, the seat drops by $0.34 \exp[-x^2/0.45 - (y+0.25)^2/0.25]$ and Y shifts by $0.25z$. Storage is 25 x 41 = 1025 vertices and 5760 indices; normals accumulate negated face cross products. Rails lift their shared collision endpoints to $z(y)+0.18$ and form eight corners using ±side and ±(0,0,0.18), producing 24 records and 36 indices per rail.

$$F = 1 + 0.035 \sin(11b + 7z) \sin^2 a \tag{5.21}$$

$$p = (0.94xF,\ 1.12yF,\ \max(-0.4,\ 0.68z)) \tag{5.22}$$

$$side = r_{rail}\ \text{normalize}(-(B-A)_y,\ (B-A)_x,\ 0) \tag{5.23}$$

Scene Construction and GPU Representation

Procedural factories build the cabinet, deck, legs, backbox, glass, controls and lounge objects. Each GameObject owns a movable, noncopyable Mesh rather than sharing one global GPU primitive. The VAO describes the VBO attributes; an optional EBO selects indexed triangles. The record has eleven logical floats, while actual byte size follows the GLM alignment configuration. GLAD guards permit CPU-only scene construction for tests and export; texture pixels are shared immutably with ray workers.

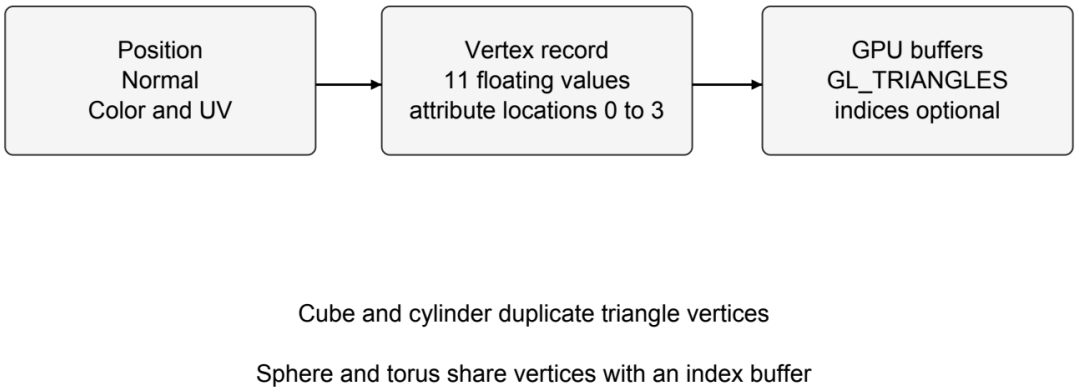


Figure 6.1: Vertex attributes, buffer storage and the two drawing paths.

Table 6.2: Vertex attributes and GPU bindings.

Location	Attribute	Components	Role
0	position	3 floats	Local surface point
1	normal	3 floats	Surface orientation
2	color	3 floats	Per-vertex color
3	texCoord	2 floats	Texture sampling coordinate

Hierarchy and Flipper Kinematics

Each mesh is centered locally, so the flipper model translates to its pivot, rotates about Z and offsets by ±length/2. The left extends along +X and the right along -X. Opposing rest angles of -28°/+28° advance to +20°/-20° at 720°/s: a 48° stroke takes approximately 0.066667 seconds.

$$M_L = T(p_L)\, R_z(\theta_L)\, T(\ell/2,0,0)\, S \tag{7.1}$$

$$M_R = T(p_R)\, R_z(\theta_R)\, T(-\ell/2,0,0)\, S \tag{7.2}$$

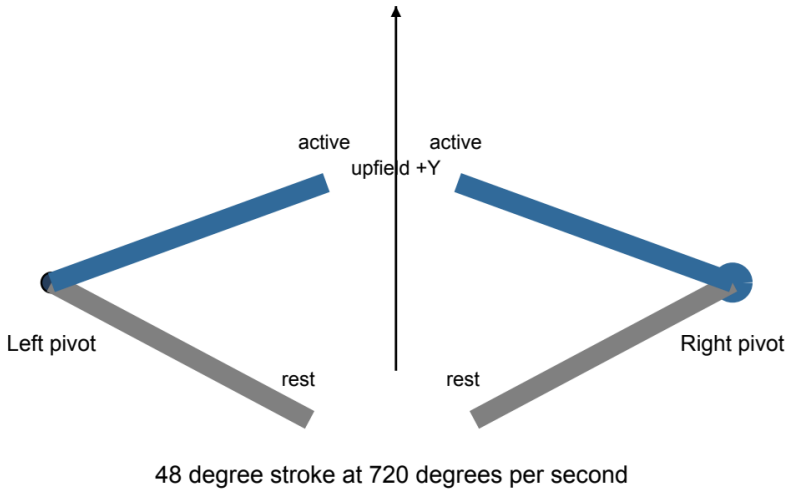


Figure 7.1: Opposing flippers at their rest and active angles.

Machine, Fan and Door Transform Chains

Machine-space component transforms remain authoritative beneath the lounge placement. The left/right contact directions are (cosθ,sinθ) and (-cosθ,-sinθ); tip = pivot + length x direction. Four fan blades share one angle with quadrant offsets. Door frame, glass and handle share stored offsets from hinge (-5.155,5.50,1.60). Its closed center offset is (0.495,0,0); opening to 100° gives center approximately (-5.240956,5.012520,1.600000) in room coordinates.

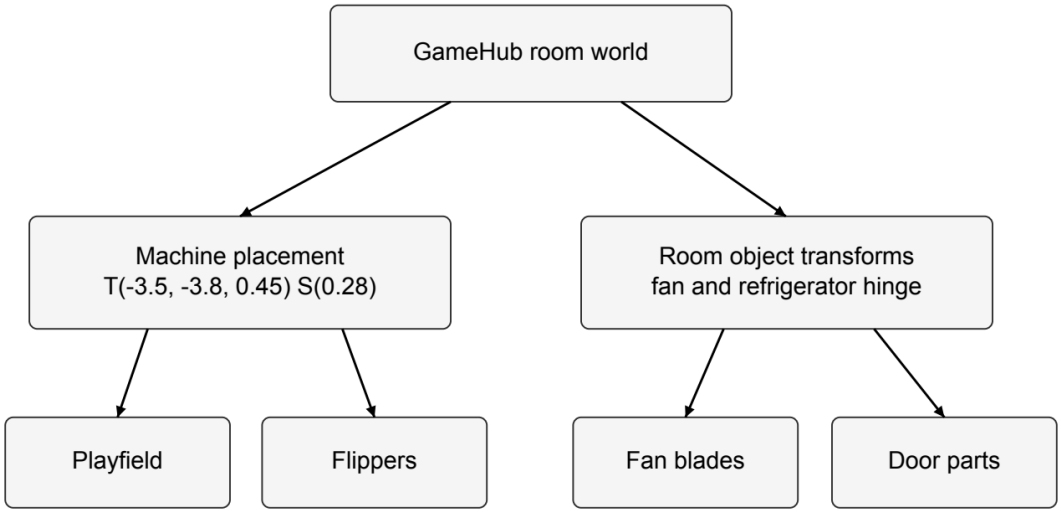


Figure 7.2: Machine placement and the independent animated room assemblies.

$$\text{bladeCenter}_i = c_{fan} + 0.77(\cos(\theta+90°i), \sin(\theta+90°i), 0) \tag{7.3}$$

$$\text{doorPartWorld} = h_{door} + R_z(-\theta_{door})\,\text{offset}_{part} \tag{7.4}$$

Ball Dynamics and Gameplay

READY holds the ball in the lane; launch changes it to SHOOTER. Crossing X < 2.14 and Y > 4 enters IN_PLAY, the only scoring state. Space requests speed 18. Holding Enter charges compression at 1.6 per second; release launches at a speed clamped to 14-20. Active or game-over launches are ignored. A ball rolling back into the lane returns to READY; R restarts.

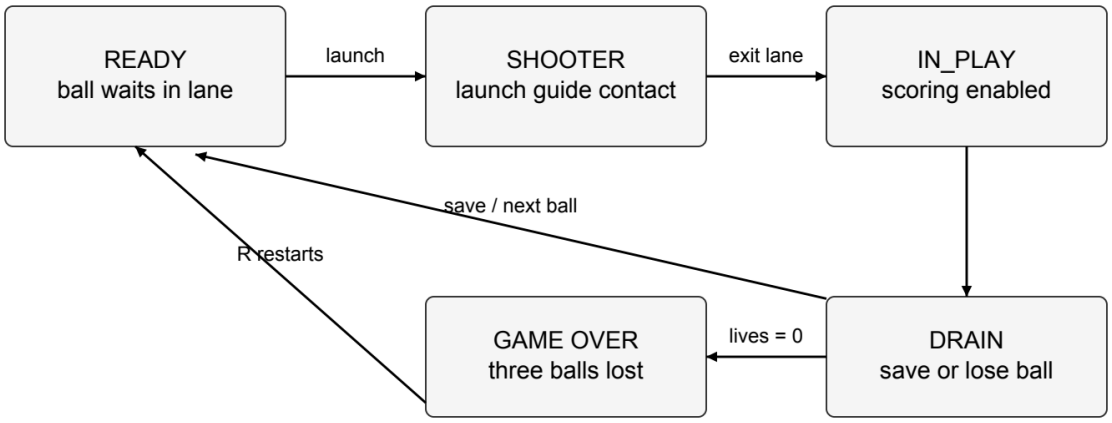


Figure 8.1: Ball launch, play, drain and restart state transitions.

$$v_{launch} = 14 + 6c \; ; \; 0 \leq c \leq 1 \tag{8.1}$$

$$plungerY = -5.85 - 0.55c \tag{8.2}$$

Fixed-Step Integration and Surface Constraint

The accumulator advances physics at $\Delta t = 1/240$ s; invalid time is rejected and each frame contributes at most 0.25 s. Projected XY velocity receives downhill acceleration, time-adjusted damping and a speed cap of 22 before position integration. After contact resolution, seatBall restores Z from the slope. This constrained model does not simulate free flight, rolling spin or full rigid-body inertia.

$$v_y \leftarrow v_y - 9.81 \sin \alpha \cos \alpha \, \Delta t \tag{8.3}$$

$$a_y = -9.81 \sin 8° \cos 8° \approx -1.352001 \tag{8.4}$$

$$v \leftarrow v \, (0.999)^{60\Delta t} \; ; \; p \leftarrow p + v\Delta t \tag{8.5}$$

$$z_{center} = 1.4 + y \tan 8° + 0.28 \; ; \; v_z = 0 \tag{8.6}$$

Scoring and Contact Guards

Incoming bumper contacts require normal speed below -0.05 and an expired 0.18-second timer. Standing targets score once before dropping; slingshots have independent 0.25-second cooldowns. The orbit arms at X < -1.9, -1.8 < Y < 2.4 and upward speed above 1, completes at Y > 4.35 and X < -0.8, and disarms below Y = -2.2. Score is capped at 99,999,999; the session best survives R but is not saved after exit.

Table 8.1: Scoring rules before applying the current multiplier.

Event	Base score	Added bonus	Guard
Left/right main bumper	100	10	0.18 s hit timer and incoming contact
Upper-middle bumper	150	10	Same bumper guard
Small side bumper	50	10	Same bumper guard
Drop target	100	50	Target must be standing
Slingshot	10	0	0.25 s cooldown per side
Left orbit	250	100	Upward traversal; 1.5 s cooldown
Four-target bank	1000	0	All four targets down
Flipper or ordinary wall	0	0	No scoring event

$$\text{scoreIncrement} = \text{basePoints} \times \text{currentMultiplier} \tag{8.7}$$

Bonus, Ball Save and Lives

Completing the four-target bank awards 1000 using the existing multiplier, then raises it toward 5x and resets the bank after 1.25 s. At 1x the targets and bank total 1400; their 200 pending bonus collects as 400 at the next unsaved drain with 2x, reaching 1800. The first launch of each ball enables an eight-second, one-use save. A saved drain returns the same ball; an unsaved drain collects bonus, resets multiplier and consumes a ball. Three losses end the game.

$$m_{new} = \min(5, m_{old} + 1) \tag{8.8}$$

$$\text{drainBonus} = \text{pendingBonus} \times \text{currentMultiplier} \tag{8.9}$$

Lighting

Phong lighting uses normalized surface-to-light and surface-to-camera vectors. The reflection vector controls the specular term, while $N \cdot L$ controls diffuse response. Shininess is bounded below by one.

$$L = \text{normalize}(\text{lightPosition} - p) \; ; \; V = \text{normalize}(\text{cameraPosition} - p) \tag{9.1}$$

$$R = \text{reflect}(-L, N) = 2(N \cdot L)N - L \tag{9.2}$$

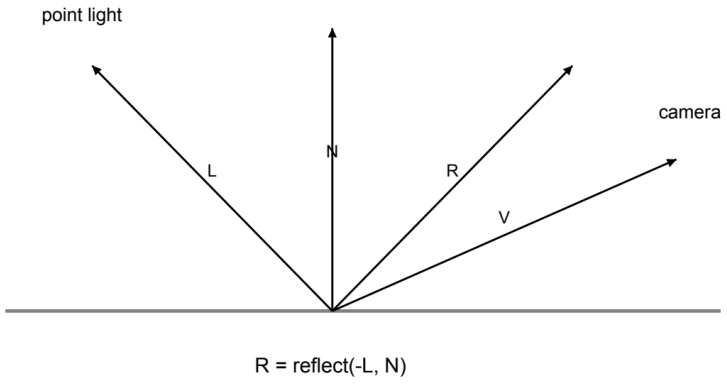


Figure 9.1: Vectors used by the project's Phong diffuse and specular calculation.

$$f_d = \max(N \cdot L,0)\ ;\ f_s = [\max(V \cdot R,0)]^{\max(s,1)} \tag{9.3}$$

Color, Attenuation and Spotlight

The point-light expression uses intensity I, distance attenuation A, albedo a and component-wise RGB multiplication ⊙. Raster lighting tints the specular term with albedo; ray lighting does not. The table spotlight uses inner/outer angles 30°/45° and exponent 20, giving shaping power one. The cone weights diffuse and specular; spotlight ambient remains outside it.

$$C_{point} = A\ I\ a \odot (L_a + L_d f_d + L_s f_s) \tag{9.4}$$

$$A(d) = 1 / (k_c + k_l d + k_q d^2) \tag{9.5}$$

$$q = L \cdot \text{normalize}(-\text{spotDirection}) \tag{9.6}$$

$$c = \text{clamp}((q - \cos\theta_{outer}) / \max(\cos\theta_{inner} - \cos\theta_{outer},0.0001),0,1) \tag{9.7}$$

$$c_{shaped} = c^{\max(0.05\ \text{exponent},0.2)} \tag{9.8}$$

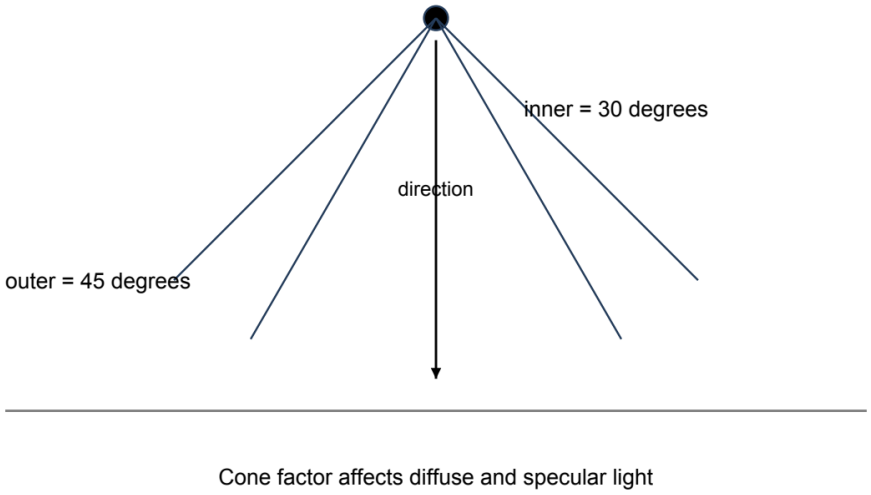


Figure 9.2: Inner and outer cone limits for the table spotlight.

Effective Light Rig and Room Controls

GameHub supplies seven effective point lights for both lounge and machine; its render path suppresses the standalone machine light setup. Point attenuation is kc = 1, kl = 0.045, kq = 0.025, with ambient 0.045 times diffuse and white specular. Turning room lighting off disables lights 0-3 while retaining refrigerator, machine and window contributions. The table spotlight at (-3.5,-3.8,3.55), directed (0,0,-1), uses intensity 2 and attenuation (1,0.05,0.02); table view or room-light power enables it. Emissive fixture surfaces do not act as sampled diffuse area lights.

Table 9.1: Effective room point-light configuration.

ID	Room position	RGB diffuse	Intensity
0	(1.0,2.8,4.7)	(0.79,0.90,1.00)	3.20
1	(-3.2,5.2,4.75)	(0.88,0.95,1.00)	2.50
2	(2.0,5.9,3.9)	(0.11,0.36,1.00)	1.50
3	(4.9,5.95,1.1)	(0.24,0.09,1.00)	1.15
4	(-4.66,5.25,1.9)	(0.50,0.74,1.00)	0.45
5	(-3.5,-3.8,1.85)	(0.80,0.88,1.00)	1.10
6	(5.2,2.5,3.9)	(0.52,0.70,1.00)	0.25 night; 2.20 day

Day, Night and Light-State Comparison

Keys 1, 3 and 4 control room-light power, day and night. Fan power is independent. Ambient RGB is (0.32,0.37,0.42) for lights-on day, (0.22,0.27,0.34) for lights-on night, (0.12,0.16,0.22) for lights-off day and (0.018,0.026,0.050) for lights-off night. Retained local lights remain visible when the main room fixtures are off.



Figure 9.3: Room lighting enabled in the Phong raster view.

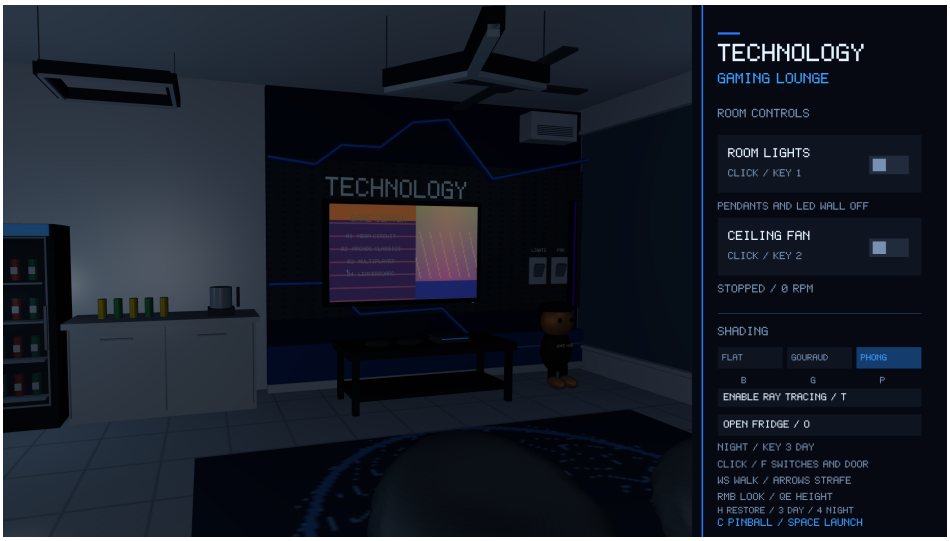


Figure 9.4: Actual room capture with room lights disabled.

Materials and Shading

Base color, alpha, emission and shininess specialize the generated meshes. Reflectivity and refractive index control ray transport. Raster opacity uses blending; glass transmission is 1-alpha in the ray scene. Opaque reflectivity mixes local lighting and recursive reflection, while glass uses Fresnel reflectance. The steel ball has shininess 96, emission 0.12 and reflectivity 0.72; window glass has shininess 128, alpha 0.12 and refractive index 1.5.

Flat, Gouraud and Phong Programs

Flat: the named program evaluates ambient and diffuse per fragment without specular. It has interpolated normals rather than true constant per-face shading. **Gouraud:** vertex lighting is interpolated, multiplied by vertex/texture color and tone-mapped; narrow highlights can be missed between coarse vertices. **Phong:** interpolated normals are renormalized and illumination is evaluated per fragment, resolving specular detail more accurately. Exposure/tone mapping below operate component-wise; the implementation has no explicit gamma conversion.

$$\text{radiance} = 0.75(\text{result} + \text{albedo} \times \text{emission}) \tag{10.1}$$

$$\text{displayRGB} = \text{radiance} / (1 + \text{radiance}) \tag{10.2}$$

Shading Comparison



Figure 10.1: Mode labeled Flat with ambient and diffuse lighting.



Figure 10.2: Gouraud mode with lighting evaluated at vertices.



Figure 10.3: Phong mode with lighting evaluated per fragment.

Normal Transformation

Nonuniform scaling requires the inverse transpose of the combined scene/object linear transform. If $n \cdot t = 0$, then $(A^{-T}n) \cdot (At) = n \cdot t = 0$, preserving perpendicularity. Vertex shaders and ray export use the same rule. Ray export separately constructs geometric normals for entry/exit classification, aligns them with shading-normal sums and discards degenerate triangles.

$$A = (M_{scene}M_{object})_{3x3} \; ; \; N_{matrix} = (A^{-1})^{T} \tag{11.1}$$

$$N_{world} = \text{normalize}(N_{matrix}N_{local}) \tag{11.2}$$

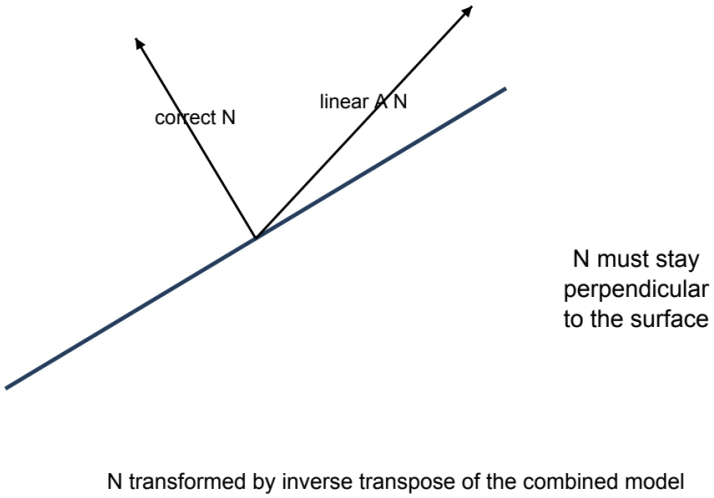


Figure 11.1: Correct and incorrect normal directions after a nonuniform transform.

Camera System

A Z-up movable camera derives its basis from yaw ψ and pitch ϕ. Room defaults are position (-2,-2.5,2.7), yaw 74°, pitch -3° and FOV 56°. Table defaults are machine position (0,-8.4,8.1), yaw 90°, pitch -46° and FOV 64°, becoming (-3.5,-6.152,2.718) in the lounge. Position is clamped to $X \in [-5.75,5.75]$, $Y \in [-6.75,6.75]$, $Z \in [0.25,5.35]$; this bounds the room rather than avoiding furniture.

$$\text{front} = \text{normalize}(\cos\psi \cos\phi, \sin\psi \cos\phi, \sin\phi) \tag{12.1}$$

$$\text{right} = \text{normalize}(\text{front} \times \text{worldUp}) \; ; \; \text{up} = \text{normalize}(\text{right} \times \text{front}) \tag{12.2}$$

$$V = \text{lookAt}(\text{position}, \text{position} + \text{front}, \text{up}) \tag{12.3}$$

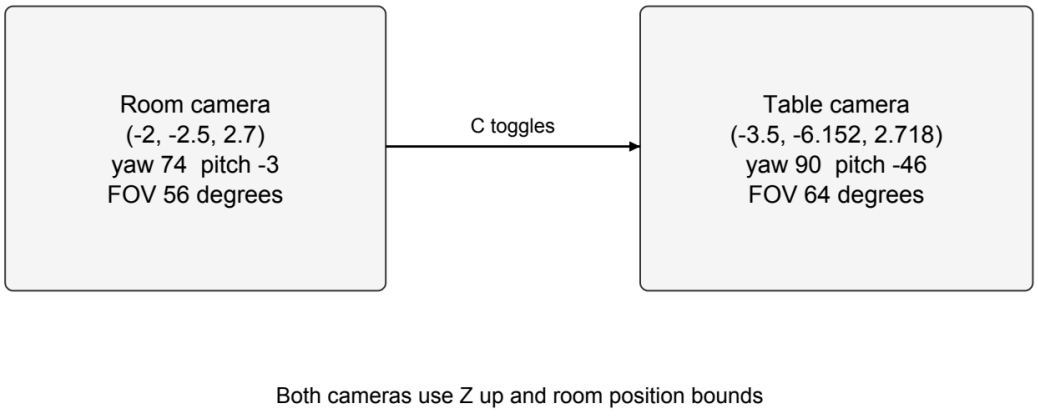


Figure 12.1: Default room and table camera states after machine placement.

Projection, Movement and Picking Rays

Perspective uses near/far planes 0.1/400 and the 3D viewport aspect, approximately 73% of framebuffer width. W/S move along front, arrows strafe, and Q/E move along Z; distance is speed x frame time. Right-drag sensitivity is 0.1° per pixel, pitch is limited to ±89°, and scroll clamps FOV to 25°-75°. Picking applies inverse projection, sets eye.z = -1 and eye.w = 0, then applies inverse view. Window-to-framebuffer scaling supports high-resolution displays. Raster and ray cameras share the basis and FOV.

$$q = 1 / \tan(\text{FOV}_y/2) \; ; \; a = \text{viewportWidth} / \text{viewportHeight} \tag{12.4}$$

$$P = \begin{bmatrix} q/a & 0 & 0 & 0 \\ 0 & q & 0 & 0 \\ 0 & 0 & -(f+n)/(f-n) & -2fn/(f-n) \\ 0 & 0 & -1 & 0 \end{bmatrix} \tag{12.5}$$

$$\text{clip} = (2x/\text{viewWidth} - 1, \; 1 - 2y/\text{height}, \; -1, \; 1) \tag{12.6}$$

Time-Based Animation

Fan speed ramps to 240°/s (40 RPM) at 120°/s^2 and coasts down at 160°/s^2, taking approximately two seconds to start and 1.5 seconds to stop. Four blades orbit center (2.3,4.1,5.07) at radius 0.77 and 90° spacing. The refrigerator door targets 0° or 100° at 100°/s. Toggling reverses its current motion continuously; all parts rotate by -θ around the shared hinge.

$$\omega \leftarrow \omega + \mathrm{clamp}(\omega_{target} - \omega, -a\Delta t, a\Delta t) \tag{13.1}$$

$$\theta \leftarrow (\theta + \omega\Delta t) \bmod 360° \tag{13.2}$$

$$\theta_{door} \leftarrow \theta_{door} + \mathrm{clamp}(\theta_{target} - \theta_{door}, -100\Delta t, 100\Delta t) \tag{13.3}$$

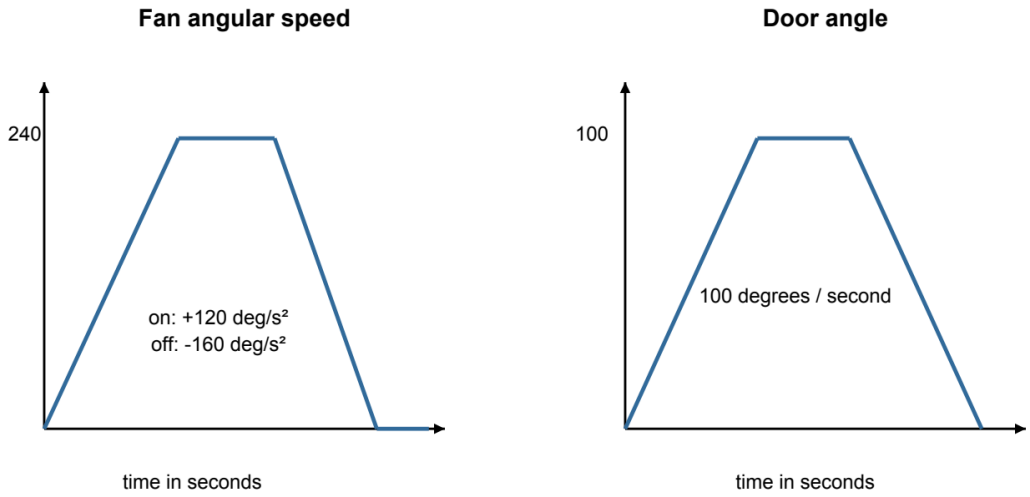


Figure 13.1: Idealized time profiles defined by the fan and door update rules.

Game Animation and Feedback

Flippers clamp at the stroke limits and derive angular contact velocity from their actual step. Bumpers flash yellow for 0.18 s and increase lamp emission. Dropped targets immediately translate down 0.18 and rotate 70° about X; the bank timer controls their later reset. Plunger compression, ball position and ray snapshots use the same state. Ball rolling has no explicit spin angle.

$$\Delta\theta = \mathrm{clamp}(\theta_{target} - \theta, -720\Delta t, 720\Delta t) \tag{13.4}$$

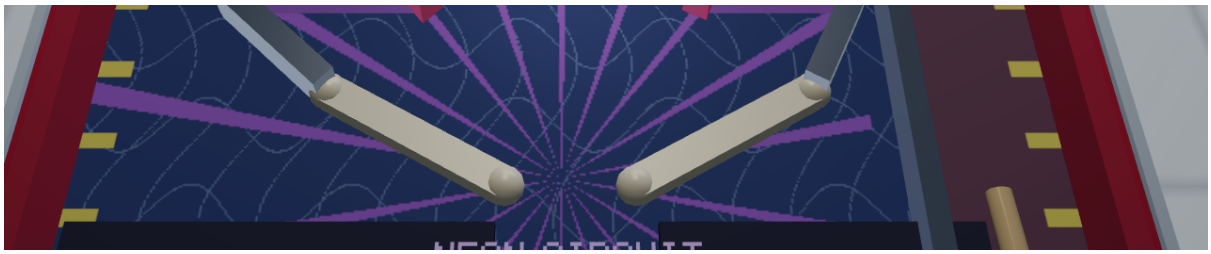


Figure 13.2: Flippers and lower-board artwork from the actual game capture.

Collision Detection and Response

Contacts operate in XY. Circular bumpers correct overlap and reflect incoming normal velocity; coincident centers use fallback normal (0,1). Ball restitution is 0.72. Valid hits add outward impulses 8.5/6.5 for main/side bumpers. Targets use an axis-aligned closest-point rectangle with half-size (0.22,0.08), fallback normal (0,-1), and skip collision once down.

$$n = (P-C)/|P-C| \text{ ; overlap} = r+R-|P-C| \tag{14.1}$$

$$P \leftarrow P + \text{overlap } n \tag{14.2}$$

$$v' = v - (1+e)(v \cdot n)n \text{ ; only if } v \cdot n < 0 \tag{14.3}$$

$$Q = C + \text{clamp}(P-C, -\text{halfSize}, \text{halfSize}) \tag{14.4}$$

Finite Rails and Moving Flippers

Rail contact projects onto a clamped segment, tests the summed radii and corrects penetration with 0.0001 extra separation. Incoming velocity changes by -(1+bounce)(v·n)n. Flippers use a 1.48-long pivot segment and contact radius rball + half-width. Their moving surface sets relative impact velocity; response coefficient 1.6 corresponds to restitution 0.6. Powered strokes enforce Y speed of at least 8 + 8t and add a side-dependent X kick of magnitude 2t.

$$t = \text{clamp}(((P-A) \cdot (B-A)) / |B-A|^2, 0, 1) \tag{14.5}$$

$$Q = A + t(B-A) \text{ ; contact if } |P-Q| < r_{ball} + r_{rail} \tag{14.6}$$

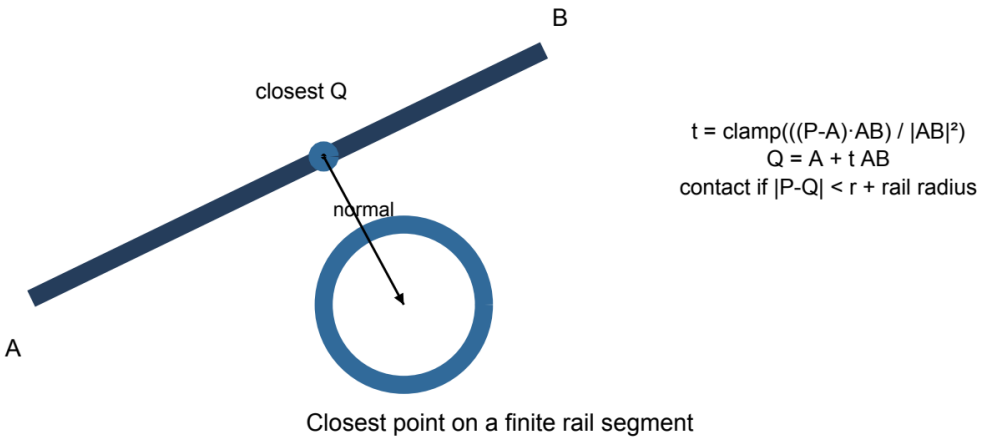


Figure 14.1: Closest-point contact between the ball and a finite rail segment.

$$v_{surface} = \omega \times r = (-\omega r_y, \omega r_x) \tag{14.7}$$

$$v_{relative} = v_{ball} - v_{surface} \tag{14.8}$$

Walls, Drain and Numerical Safeguards

Side limits are ±(3.5-0.11-0.28) = ±3.11; the upper limit is 6-0.11-0.28 = 5.61. Wall restitution is 0.7 for X and 0.8 at the top. The bottom stays open. Discrete 240 Hz contact reduces tunneling risk but is not continuous collision detection. Zero-length rails below squared length 10^{-8} are skipped; coincident contacts use deterministic normals, and invalid time is rejected. Flipper kicks reapply the speed cap; bumper impulses can exceed it until the next step.

$$\text{drain condition: ballY} < -6 - 0.28 = -6.28 \tag{14.9}$$

Lounge Layout and Ray Tracing

The lounge spans a 12 x 14 floor with height 5.6. Machine placement (-3.5,-3.8,0.45) leaves the central rug and television area open. Procedural objects, movable assemblies and glass supply a larger scene for lighting, picking, reflection and refraction.

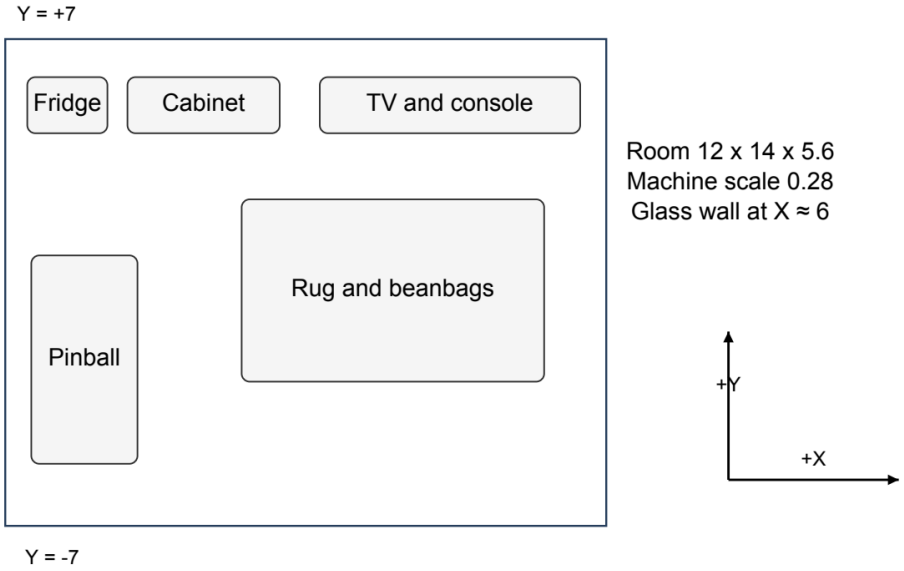


Figure 15.1: Schematic plan of the gaming lounge and machine placement.

Scene Snapshot and Bounding Volume Hierarchy

GameHub::rayScene exports world-space triangles and normals into a self-contained snapshot; immutable texture pixels keep workers independent of live objects and OpenGL. The documented startup export contains 27,870 nondegenerate triangles, 83,610 corner records, 322 materials and 16,383 BVH nodes. Leaves hold at most four triangles. The builder median-splits on the largest centroid extent; near-first traversal shortens the maximum distance after closer hits.

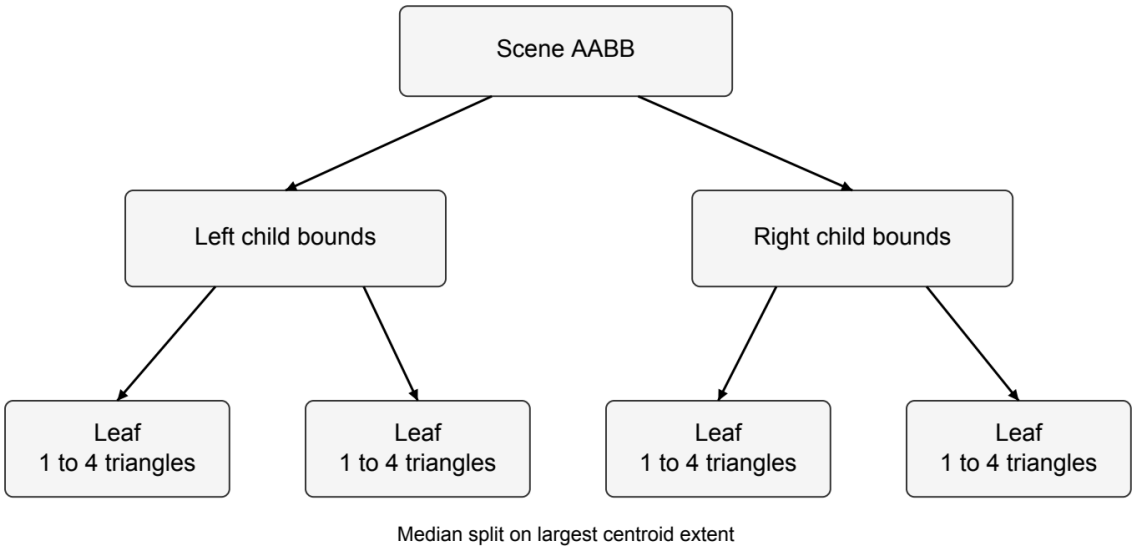


Figure 15.2: Bounding volume hierarchy used to accelerate triangle intersection.

$$\text{centroid} = (v_0 + v_1 + v_2)/3 \tag{15.1}$$

Ray Intersections and Hit Attributes

A normalized ray $r(t) = o + td$ intersects AABB slab intervals, with near-parallel axes handled separately. The two-sided Möller-Trumbore test uses $e_1 = v_1\text{-}v_0$ and $e_2 = v_2\text{-}v_0$, rejects $|D| < 10^{-10}$, and accepts $0 \le u \le 1$, $0 \le v$, $u+v \le 1$ within the permitted distance. Barycentric weights interpolate normal, RGB and UV; normals are renormalized, position is $o + td$, and the texture sampler provides albedo.

$$t_{axis,0} = (min_{axis}\text{-}o_{axis})/d_{axis} \; ; \; t_{axis,1} = (max_{axis}\text{-}o_{axis})/d_{axis} \tag{15.2}$$

$$p = d \times e_2 \; ; \; D = e_1 \cdot p \; ; \; s = o\text{-}v_0 \tag{15.3}$$

$$u = (s \cdot p)/D \; ; \; q = s \times e_1 \; ; \; v = (d \cdot q)/D \tag{15.4}$$

$$t = (e_2 \cdot q)/D \; ; \; \text{barycentric weights} = (1\text{-}u\text{-}v, u, v) \tag{15.5}$$

$$\text{attribute} = (1\text{-}u\text{-}v)\text{attribute}_0 + u\,\text{attribute}_1 + v\,\text{attribute}_2 \tag{15.6}$$

Shadows and Mirror Reflection

Direct Phong lighting uses a bounded shadow segment for each point/spot light. Opaque hits block it; transparent hits tint and reduce visibility through up to 32 layers. The approximation follows straight shadow paths and uses ambient rather than diffuse indirect transport. Mirror rays see actual geometry beyond the camera view; reflectivities include ball 0.72, plunger 0.40, floor 0.18 and kettle 0.50. New origins offset by $\varepsilon = 0.0001$ along the outgoing geometric-normal side.

$$\text{shadow segment: } \varepsilon \le t < |\text{lightPosition}-p| - \varepsilon \tag{15.7}$$

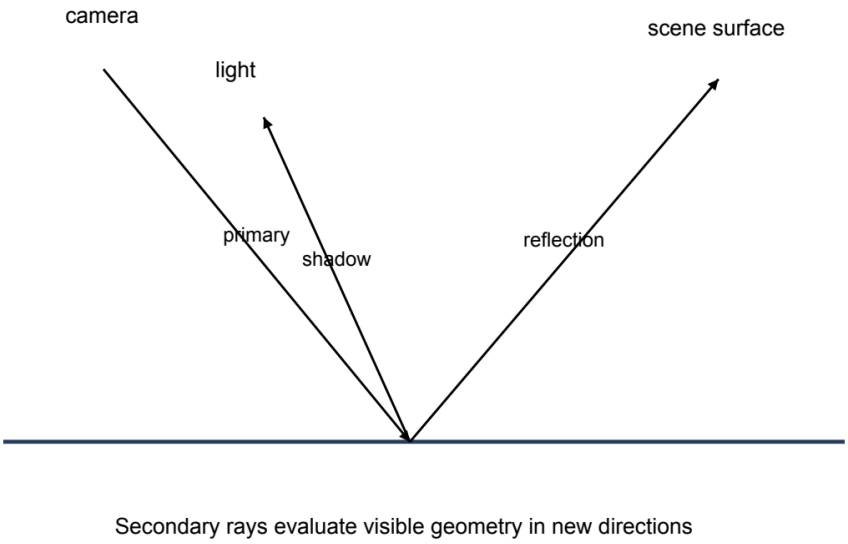


Figure 15.3: Primary, bounded shadow and recursive reflection rays.

$$d_{reflection} = d - 2(d \cdot N)N \tag{15.8}$$

Refraction, Fresnel and Total Internal Reflection

Glass uses refractive index 1.5. Geometric normals distinguish air entry and exit; Snell refraction and the exact average of squared parallel/perpendicular Fresnel amplitudes divide transport. Normal-incidence reflectance is $((1-1.5)/(1+1.5))^2 = 0.04$. When $\eta \sin\theta_1 \geq 1$, $F = 1$ and transmission stops; the glass-air critical angle is $\text{asin}(1/1.5) \approx 41.81°$. Recursion stops after six secondary levels or contribution ≤ 0.005. Nested media and volumetric absorption are excluded.

$$n_1 \sin\theta_1 = n_2 \sin\theta_2 \tag{15.9}$$

$$\eta = n_1/n_2 \; ; \; d_{transmitted} = \text{refract}(d,N,\eta) \tag{15.10}$$

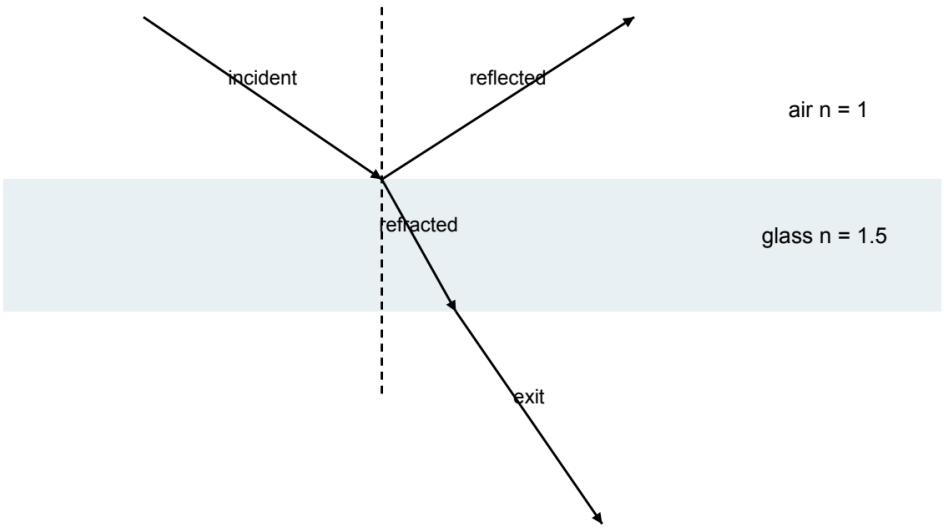


Figure 15.4: Reflection and refraction at the two interfaces of a glass pane.

$$F = (r_{parallel}^{2} + r_{perpendicular}^{2})/2 \tag{15.11}$$

$$weight_{transmitted} = (1\text{-}F)\ \sqrt{transmission} \tag{15.12}$$

Ray-Traced Results

These original executable captures use a ray width of 960 and four samples per pixel. The closed and open door views export the matching hinge state for glass, frame and handle.



Figure 15.5: Fresh room ray-tracing capture at a 960-pixel ray width and four samples per pixel.



Figure 15.6: Actual ray-tracing capture with the refrigerator door open.

Background Rendering and Quality Controls

RayTraceView uploads completed CPU RGB pixels into an OpenGL texture beneath the HUD. Live preview defaults to width 256 and one sample; captures default to width 480 and four samples. Width is clamped to 64-1920 and samples to 1-64. Up to eight workers claim rows atomically and use deterministic sample offsets, producing identical results across worker counts. Camera, viewport and control changes invalidate obsolete jobs; raster output appears while no current result is ready. Completed snapshots may lag animation. This is a CPU Whitted-style renderer with direct lighting and ideal mirror/dielectric recursion.

$$\text{rayImageHeight} = \text{round}(\text{rayWidth} \times \text{viewportHeight} / \text{viewportWidth}) \quad (15.13)$$

Interaction and Runtime Controls

One-shot controls are edge-latched; flipper and plunger inputs remain continuous while held. The runtime controls below provide both gameplay and scene demonstrations.

Table 16.1: Runtime keyboard and mouse controls from main.cpp.

Input	Action
Space	Launch a READY ball at speed 18; switch to table view for a valid launch.
Enter / keypad Enter	Hold to charge plunger; release to launch a READY ball.
A / D	Power left / right flipper while held.
R	Reset score, lives, bonus, bank and ball; preserve session best.
C	Toggle room and table cameras.
W / S	Move active camera forward / backward.
Left / right arrows	Strafe active camera left / right.
Q / E	Move active camera upward / downward.
Right mouse drag	Look around with active camera.
Scroll	Change active camera field of view.
H	Restore default room composition and select room camera.
1 / 2	Toggle room lighting / fan power.
3 / 4	Select day / night mode.
B / G / P	Select Flat / Gouraud / Phong raster mode.
T	Toggle CPU ray tracing.
O	Toggle refrigerator-door target between closed and open.
F	Use the crosshair ray to activate a room switch or door.
Left click	Activate room console button or pick room switches/door.
Escape	Exit the application.

Console Buttons and Scene Picking

Console hit-testing precedes scene picking. A 320 x 720 design space scales into the approximately 27%-wide console; window clicks convert to framebuffer pixels and then design coordinates. Switches use room-space slab boxes. Door picking applies its inverse model, with origin w = 1 and direction w = 0. Room picking is disabled in table view. The consoles report game progress and light, fan, shader, door and ray-time states.

$$s = \min(0.27W/320, H/720) \tag{16.1}$$

Rendering Flow

The frame resets depth/blend state, clears the target, selects a shader and uploads camera matrices for the 3D viewport. GameHub draws room objects and the placed machine, followed by transparent glass. The GPU clips, rasterizes and shades triangles; the final full-window viewport draws the HUD.

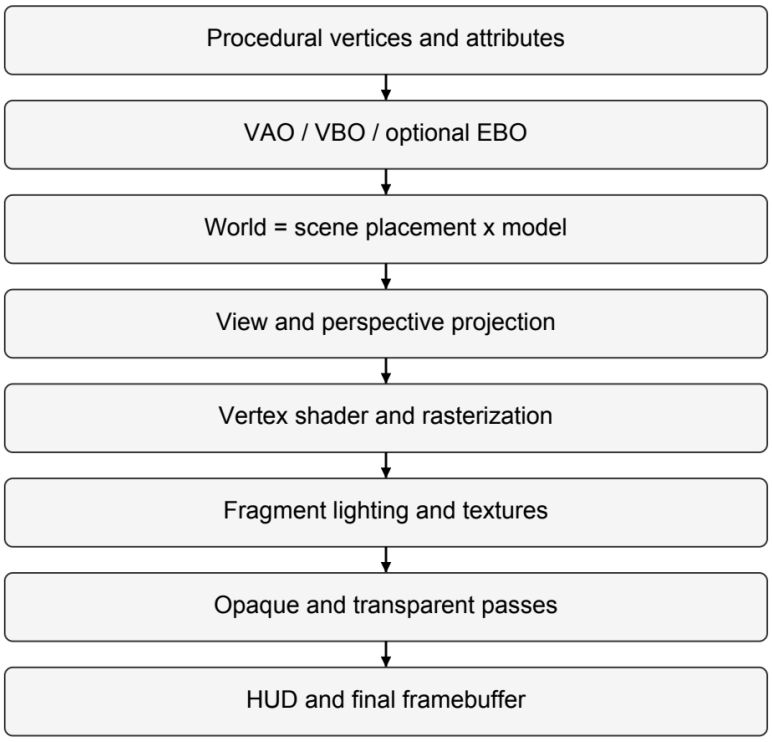


Figure 17.1: Raster pipeline from generated vertices to the final framebuffer.

$$p_{clip} = P\ V\ M_{scene}\ M_{object}\ p_{local} \tag{17.1}$$

Transparency, Textures and Ray Presentation

Opaque passes write depth. Glass enables source-alpha blending and disables depth writes; room glass sorts by descending object-center distance, then state is restored. This is not general order-independent transparency. Procedural RGB textures use 512 x 1024 board artwork and 128 x 128 room textures. Raster sampling repeats with mipmap-linear minification and linear magnification; ray sampling repeats with bilinear filtering and no mipmap pyramid. Completed ray images upload as GL_RGB8 and display on a full-screen triangle with the normal HUD. Startup requests OpenGL 3.3 core and four-sample multisampling; shaders are located beside the executable.

$$C_{blend} = \alpha C_{source} + (1-\alpha)C_{destination} \tag{17.2}$$

Implementation and Validation

Mesh.cpp constructs geometry and textures; Transform.cpp, Flipper.cpp and GameHub.cpp maintain object, pivot and hinge transforms. PinballMachine.cpp owns contact rules and game progression. Camera.cpp and main.cpp provide camera/input handling; GLSL shaders compute raster illumination. RayTracer.cpp builds and traverses the BVH and evaluates optical transport; RayTraceView.cpp manages asynchronous display, while HUD.cpp renders the consoles.

Recorded Build and Regression Results

The original report records a successful CMake build and three passing CTest suites on 8 October 2026. Physics coverage includes launch speeds 14, 18 and 20 at render intervals corresponding to 30, 60, 144 and 10 FPS, plus scoring, saves, lives and flippers. Ray tests compare BVH hits with an exhaustive reference and verify identical single-worker/four-worker RGB output. These results remain the original validation record.

Table 18.3: Observed regression-test run.

CTest suite	Result	Observed time	Main coverage
pinball_physics	PASS	1.70 s	12 launch combinations, charging, scoring, save, lives, flippers, frame-rate consistency
arena_controls	PASS	0.51 s	Switch picking, independent controls, fan coast, door reversal, bounds, HUD scaling
raytracing	PASS	1.03 s	BVH reference agreement, normals, shadows, reflection, refraction, Fresnel/TIR, deterministic workers

The recorded table capture had score 450 in IN_PLAY; the room capture had score 0 in READY. A 960 x 740, four-sample ray image over 27,870 triangles and 16,383 BVH nodes took 15,292.2 ms locally. This single capture and the suite times describe the documented run, rather than a general hardware benchmark. Reproduction commands: `cmake -S . -B build -DCMAKE_BUILD_TYPE=Release`; `cmake --build build -j4`; `ctest --test-dir build --output-on-failure`.

Coordinate and Geometry Reference

Component centers follow the shared board-height function: bumper center Z is z(y)+0.225 and target center Z is z(y)+0.22. At Y = -4.6, surface Z = 0.753512; the READY ball is (2.95,-4.6,1.033512), and flipper pivot Z = 0.873512. Rest tips are (±0.443238,-5.294818,0.873512); active tips are (±0.359255,-4.093810,0.873512). These are collider endpoints; rendered cap patches can extend beyond them.

Rail Endpoint Registry

Each endpoint is promoted to Z = 1.4 + Y tan8° + 0.18. Equation 5.23 supplies the side vector; corners use A/B ±side ±Zoffset, with Zoffset = (0,0,0.18). Drawing and collision therefore share the same XY guides.

Table A.2: All eleven physical and visible rail segments.

ID	A X,Y	B X,Y	Radius	Bounce	Type
0	(2.42,-6.00)	(2.42,4.25)	0.08	0.45	Guide
1	(3.42,4.65)	(2.35,5.78)	0.08	0.85	Guide
2	(2.35,5.78)	(0.20,5.78)	0.08	0.80	Guide

Table A.2: All eleven physical and visible rail segments.

ID	A X,Y	B X,Y	Radius	Bounce	Type
3	(-3.30,3.80)	(-2.90,5.35)	0.09	0.65	Guide
4	(-2.90,5.35)	(-1.30,5.78)	0.09	0.65	Guide
5	(-3.30,-2.40)	(-2.60,-3.90)	0.09	0.55	Guide
6	(-2.60,-3.90)	(-1.75,-4.60)	0.09	0.55	Guide
7	(2.30,-2.40)	(2.15,-3.90)	0.09	0.55	Guide
8	(2.15,-3.90)	(1.75,-4.60)	0.09	0.55	Guide
9	(-2.50,-2.45)	(-1.05,-3.95)	0.10	0.85	Sling
10	(1.65,-2.45)	(0.95,-3.95)	0.10	0.85	Sling

Complete Geometry Archive

The companion Interactive_3d_Pinball_Geometry.zip contains local primitive vertex/triangle CSVs and a complete room-world startup snapshot. Primitive columns are vertex_id, x,y,z, nx,ny,nz, r,g,b, u,v; triangle files use triangle_id and three vertex IDs. The complete scene uses triangle_id, corner 0-2, material_id and the same attributes. Repeated seam/hard-edge positions are intentional. The snapshot has fan off, door closed, ball READY and night lighting enabled. CSV data stores nine significant digits from single-precision source calculations; report values are rounded. Archive counts and schema describe the original export.

Conclusion and Future Work

The project connects procedural geometry to visible gameplay through shared transforms and state: board height seats components, rails share contact endpoints, flipper angles drive both drawing and impact response, and hinge offsets preserve the door assembly. Fixed 240 Hz physics separates contact updates from display timing. Raster and CPU ray rendering demonstrate complementary lighting and transport methods. Future work includes rolling/free-flight physics, swept contacts, true face-constant Flat shading, shared GPU meshes, furniture-aware camera collision, improved BVH splitting, progressive accumulation, rough reflections, diffuse indirect transport and nested media.